

Tabula

Rapport d'implementation

Robustesse en production

9 mars 2026

105

Tests backend

16

Tests frontend

6

Phases

0

Erreurs ruff

Resume executif

Ce rapport documente l'implementation de six axes d'amelioration pour la robustesse en production du projet **Tabula**, un outil d'extraction de tables depuis des documents PDF. Le projet supporte un double mode de fonctionnement (mono-utilisateur / multi-utilisateurs) avec authentication JWT, isolation des donnees, et partage de templates.

- **Phase 1** — Cache en memoire de la configuration applicative
- **Phase 2** — Pool de threads borne pour le traitement en arriere-plan
- **Phase 3** — Rate limiting sur les endpoints sensibles
- **Phase 4** — Migration Alembic pour le schema multi-utilisateurs
- **Phase 5** — Nettoyage de la configuration Docker
- **Phase 6** — 25 nouveaux tests d'isolation multi-utilisateurs

Resultat final : 105 tests backend passants (80 existants + 25 nouveaux), 16 tests frontend passants, linter ruff propre (0 erreurs). Aucune regression introduite.

Phase 1 — Cache de la configuration

Probleme : `get_app_config()` lisait et parsait `tabula.config.json` sur le disque a chaque requete HTTP (middleware `setup_gate` + dependances auth + operations JWT).

Solution : Variable module `_cached_config` avec invalidation explicite. L'instance `AppConfig` est immuable apres creation ; le GIL de Python garantit l'assignation atomique.

Implementation

- `get_app_config()` retourne `_cached_config` si non-None ; sinon lit le fichier, stocke, et retourne.
- `save_app_config()` ecrit le fichier ET met a jour `_cached_config`.
- `invalidate_config_cache()` remet le cache a None.
- Les valeurs par default (fichier absent) ne sont PAS cachees pour permettre la detection du fichier apres le setup wizard.

Tests ajoutes (4)

- `test_cache_returns_same_object` — deux appels retournent le meme objet (identite)
- `test_save_updates_cache` — apres save, get retourne les nouvelles valeurs
- `test_invalidate_forces_reread` — apres invalidation, relecture depuis le disque
- `test_defaults_not_cached` — quand le fichier n'existe pas, le cache reste None

- ~ `backend/app/config.py`
 - ~ `backend/tests/test_config.py`
-

Phase 2 — Pool de threads borne

Probleme : `threading.Thread` lance sans limite de concurrence pour chaque PDF uploadé. Risque d'épuisement des ressources en cas d'uploads massifs simultanés.

Solution : `ThreadPoolExecutor` avec 4 workers max. Arrêt propre via le `lifespan` FastAPI (`wait=True`, `cancel_futures=False`).

Architecture

- `worker_pool.py` : singleton paresseux avec `get_executor()` et `shutdown_executor()`
- `main.py` : `lifespan` async context manager appelant `shutdown` au stop du serveur
- `documents.py` : remplacement de `Thread(target=...).start()` par `get_executor().submit()`

Tests ajoutés (4)

- `test_get_executor_returns_pool` — vérifie le type `ThreadPoolExecutor`
- `test_get_executor_singleton` — deux appels retournent le même objet
- `test_shutdown_and_recreate` — après `shutdown`, nouveau pool créé
- `test_submit_runs_function` — soumet un callable, vérifie son exécution

- + `backend/app/core/worker_pool.py`
- + `backend/tests/test_worker_pool.py`
- ~ `backend/app/main.py`
- ~ `backend/app/api/documents.py`
- ~ `backend/tests/test_api_documents.py`

Phase 3 — Rate limiting

Probleme : Aucune protection contre les abus : force brute sur le login, spam d'upload, tentatives de setup répétées.

Solution : `slowapi` (base sur `limits`) avec decorateurs par endpoint. Cle de limitation : adresse IP distante.

Limites appliquées

Endpoint	Limite	Justification
POST /api/auth/login	10/minute	Protection force brute
POST /api/auth/register	5/minute	Anti-spam de comptes
POST /api/auth/refresh	30/minute	Limite raisonnable
POST /api/documents/upload	20/minute	Protection ressources
POST /api/setup/init	3/minute	Endpoint critique

- + `backend/app/core/rate_limit.py`

- ~ backend/app/main.py
- ~ backend/app/api/auth.py
- ~ backend/app/api/documents.py
- ~ backend/app/api/setup.py
- ~ backend/requirements.txt

Phase 4 — Migration Alembic

Problème : Migration manquante pour la table `users` et les colonnes `user_id` / `is_shared` ajoutées pour le support multi-utilisateurs.

Solution : Migration Alembic créant la table `users`, ajoutant les FK `user_id` sur `documents` et `templates`, et la colonne `is_shared` sur `templates`.

Schema de la table `users`

Colonne	Type	Contraintes
<code>id</code>	UUID	Cle primaire
<code>username</code>	String(150)	Unique, index
<code>password_hash</code>	String(255)	bcrypt hash
<code>is_admin</code>	Boolean	Default : false
<code>is_active</code>	Boolean	Default : true
<code>created_at</code>	DateTime	Default : now()

Note : Cette migration ne concerne que PostgreSQL. En mode SQLite, les tables sont créées via `Base.metadata.create_all()` dans `db_init_service.py`.

- + backend/alembic/versions/a1b2c3d4e5f6_add_users_...py
- ~ backend/alembic/env.py

Phase 5 — Nettoyage Docker

Problème : `docker-compose.yml` référençait `DATABASE_URL` qui n'est plus utilisé depuis le passage au système de configuration par fichier JSON (`tabula.config.json`).

Solution : Suppression de `DATABASE_URL`, ajout de `DATA_DIR` et d'un volume `appdata` pour persister la config. Script d'entrypoint avec migration Alembic conditionnelle.

Script d'entrypoint

- Si `$DATA_DIR/tabula.config.json` existe et que `mode == "multi"` : exécute `alembic upgrade head`
- Sinon : démarrage direct (le setup wizard gère l'initialisation)
- Dans tous les cas : `exec uvicorn app.main:app --host 0.0.0.0 --port 8000`

```
+ docker-entrypoint.sh
~ docker-compose.yml
~ Dockerfile
~ .env.example
```

Phase 6 — Tests multi-utilisateurs

Probleme : Les 80 tests existants tournaient exclusivement en mode mono-utilisateur. Aucun test ne verifiait l'isolation des donnees, le partage de templates, ni l'administration en mode multi.

Solution : 25 nouveaux tests repartis dans 4 fichiers, avec fixtures dediees (multi_config, admin_user, regular_user, second_user, tokens JWT, multi_client).

test_document_ownership.py (6 tests)

- Isolation : alice upload, bob liste = vide
- Acces interdit : bob GET/DELETE doc d'alice = 404
- Admin voit et supprime tout
- Requete non-authentifiee = 401

test_template_sharing.py (6 tests)

- Isolation : alice cree, bob ne voit pas
- Template partage visible par tous
- Non-propretaire ne peut pas modifier/supprimer
- Admin peut toggler le partage ; non-admin = 403

test_admin.py (8 tests)

- CRUD utilisateurs (admin ok, non-admin = 403)
- Admin ne peut pas se supprimer lui-meme
- Toggle allow_registration (admin ok, non-admin = 403)

test_mode_switch.py (5 tests)

- Mono vers multi : mock services, verification config
- Multi vers mono : idem, sens inverse
- Cas d'erreur : deja dans le mode cible, connexion echouee

```
+ backend/tests/test_document_ownership.py
+ backend/tests/test_template_sharing.py
+ backend/tests/test_admin.py
+ backend/tests/test_mode_switch.py
~ backend/tests/conftest.py
```

Probleme technique resolu

Contamination entre tests

Symptome : 19 tests mono-mode echouaient (401/403/422) lorsqu'ils s'executaient apres les tests admin multi-mode.

Cause racine :

- from app.config import is_multi_user cree une **reference locale** dans chaque module importeur.
- Patcher app.config.is_multi_user n'affecte PAS ces references locales dans dependencies.py, auth.py, etc.
- Les tests admin appelaient save_app_config() qui ecrivait un fichier config sur disque avec mode="multi", polluant les tests mono suivants.

Solution :

Patcher aux **sites d'import** plutot qu'au site de definition :

```
patch("app.core.dependencies.is_multi_user", return_value=False)
patch("app.api.auth.is_multi_user", return_value=False)
patch("app.main.is_setup_completed", return_value=True)
```

Bilan des fichiers

8 fichiers crees, 14 fichiers modifies

Fichier	Action	Phase
backend/app/config.py	Modifie	1
backend/app/core/worker_pool.py	Cree	2
backend/app/core/rate_limit.py	Cree	3
backend/app/main.py	Modifie	2, 3
backend/app/api/documents.py	Modifie	2, 3
backend/app/api/auth.py	Modifie	3
backend/app/api/setup.py	Modifie	3
backend/requirements.txt	Modifie	3
backend/alembic/env.py	Modifie	4
backend/alembic/versions/a1b2...py	Cree	4
docker-compose.yml	Modifie	5
Dockerfile	Modifie	5
docker-entrypoint.sh	Cree	5
.env.example	Modifie	5
backend/tests/conftest.py	Modifie	6

backend/tests/test_config.py	Modifie	1
backend/tests/test_api_documents.py	Modifie	2
backend/tests/test_worker_pool.py	Cree	2
backend/tests/test_document_ownership.py	Cree	6
backend/tests/test_template_sharing.py	Cree	6
backend/tests/test_admin.py	Cree	6
backend/tests/test_mode_switch.py	Cree	6

Verification finale

Verification	Resultat	Statut
Backend (pytest -v)	105 tests passants	OK
Frontend (vitest run)	16 tests passants	OK
Lintier (ruff check)	0 erreurs	OK
Regressions	Aucune	OK